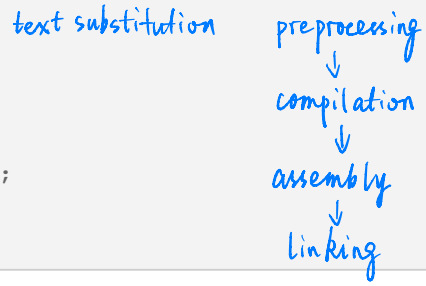
## Q1

**1 Point**

```
1: #include <stdio.h>
2: #define N 100
3: int x = N;
4: int main() {
5:   int *p = &N;
6:   *p = 10;
7:   printf("x = %d\n", x);
8:   return 0;
9: }
```




For the above C program, which of the following is true?

- It compiles and runs successfully with the printout "x = 10"
- It compiles and runs successfully with the printout "x = 100"
- It fails to compile due to line 3
- It fails to compile due to line 5
- It fails to compile due to line 6
- It compiles successfully and runs into a segmentation fault at line 5
- It compiles successfully and runs into a segmentation fault at line 6

### Q2 C program organization

**1 Point**

The following C program file `mytest.c` is added to your lab1 directory. This program parses a commandline argument as integer and then invokes `flip_bit_at_pos`. We assume you've implemented the `flip_bit_at_pos` function correctly in file `bitfloat.c`.

```
1: #include <stdio.h>
2: #include <stdlib.h>

3: int
4: main(int argc, char **argv)
5: {
6:     int a = atoi(argv[1]); //atoi is a stdlib function for turning a str into an int.
7:     int aa = flip_bit_at_pos(a, 0); //flip the least significant bit
8:     printf("%d\n", aa);
9:}
```

signature in bitfloat.h
implementation in bitfloat.c

After typing `gcc -c mytest.c`, the following error message appears:

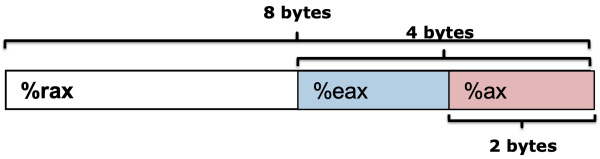
```
mytest.c: In function 'main':
mytest.c:7:12: warning: implicit declaration of function 'flip_bit_at_pos' [-Wimplicit-function-declaration]
    9 |    int aa = flip_bit_at_pos(a, 0);
```

What's the problem and how to fix it?

- The error is due to a typo in the function name and one should fix the typo.
- The error can be ignored and the correct object file is still created.
- The error is due to compiler missing information about `flip_bit_at_pos` and can be fixed by using command `gcc -c mytest.c bitfloat.c`
- ✓ The error is due to missing information about `flip_bit_at_pos` and can be fixed by adding `#include "bitfloat.h"` after line 2

## Q3 mov instruction

**1 Point**




After x86 CPU executes instruction `movq $0x12345678, %rax`, which of the following is true?

Hint: %eax refers to the lower order 4-bytes of register %rax. And %ax refers to the lower order 2-bytes of register %rax

- The higher order 4-byte of register `%rax` are all zeros.
- The higher order 4-bytes of register `%rax` remain the same as before the movq instruction is executed.
- Register `%eax` has value `0x00000000`
- Register `%eax` has value `0x12345678`
- Register `%eax` is not changed by the movq instruction.
- Register `%ax` has value `0x1234`
- Register `%ax` has value `0x5678`

## Q4 mov

**1 Point**

Suppose register `%rax` stores C variable `long *x`. Which of the following instruction corresponds to the C statement `*x = 10;`

- `movq $10, %rax`
- ✓ `movq $10, (%rax)`
- `movq (%rax), $10`
- `movq %rax, $10`

**Q5 mov**

**1 Point**

Suppose register `%eax` stores C variable `int x`. Which of the following instruction corresponds to the C statement `x = 10;`

- ✓ `movl $10, %eax`
- `movq $10, %eax`
- `movl $10, (%rax)`
- `movq $10, (%rax)`

4B

movl : move long 32b (4B)

movq : move quaadword 64b (8B)

word : 16b for historical reason

**Q6 mov**

**1 Point**

Given instruction `movl $eax, (%rbx)`, what are likely data types for the variable stored in `%rbx`? Choose all that apply.

0x1234567 8ffeeddcc

0x1234567 8ffeeddcd

0x1234567 8ffeeddce

0x1234567 8ffeeddcf

memory address of a 4B object

- long
- unsigned long
- int
- unsigned int
- ✓ int*
- ✓ unsigned int*
- long*
- unsigned long *

**Q7 Registers and C variables**

**1 Point**

4B → int / unsigned int

Suppose register %eax corresponds to the C variable x of some integer type. If the value of %eax is `0xffffffff`, what potentially could be the type and value of x?

- [x] type: int, value: -1
- [ ] type: int, value: $-2^{31}$
- [ ] type: long, value: -1
- [ ] type: long, value: $-2^{63}$
- [x] type: unsigned int, value: $2^{32} - 1$
- [ ] type: unsigned int, value $2^{32}$
- [ ] type: unsigned long, value $2^{32} - 1$
- [ ] type: unsigned long, value $2^{32}$

**Q8 Deference pointers**

**1 Point**

Suppose %rsi corresponds to C variable y of some pointer type. Which of the following instructions dereference the pointer y? Circle all that apply.

int a[8]; a[4]

a + 4 * 4

- [ ] leaq (%rsi), %rax
- [x] movq (%rsi), %rax

%rax = *y;

- [ ] movq %rsi, %rax
- [x] subq %rax, (%rsi)

*y -= %rax;

- [ ] subq %rax, %rsi
- [ ] none of the above

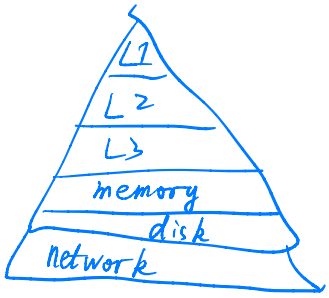
**Q9**
**1 Point**

Which of the following statements are true?

- During a program's execution, its instructions are stored on disk while its program data is stored in the memory.
- During a program's execution, both its instructions and program data are stored in the memory.
- Compilers must generate explicit instructions to increment PC (aka %rip)
- CPU automatically increments PC (aka %rip) as instructions are executed.
- An executable file compiled for ARM can be directly executed by an x86 CPU.
- An executable file compiled for ARM can not be directly executed by an x86 CPU.

**Q10 Basic machine execution**
**1 Point**

Which of the following statements are true?




- Accessing data stored in memory is as fast as accessing data stored in CPU registers.
- Accessing data stored in memory is much slower than accessing data in CPU registers.
- A C program is compiled into x86 instructions which are directly executed by the CPU.
- A Java program is compiled into x86 instructions which are directly executed by the CPU. Java byte code JVM
- One can use %rip as an operand for the `mov` instruction

not general purpose register